

AI Data Augmentor - Assignment #3 Reflection

Aleks Obuhovskiy | From spreadsheet enrichment to an agent-like, cloud-deployed workflow

50 companies

Looping agent

Live Streamlit UI

AWS + Terraform + CI + CDN

1. Objective and Final Outcome

I started with a spreadsheet of 50 outdoor and apparel companies and built a Python data augmentor to add Location, Phone, Website, and Source. The final workbook is output/augmented-companies.xlsx. The solution favors reliability over completeness: when evidence is weak, the field remains UNKNOWN instead of being guessed.

2. Core Agent-Like Workflow

The main controller is src/augment_companies.py. It reads the starter CSV, uses DDGS/DuckDuckGo Search to discover candidate sources, retrieves official pages with requests + BeautifulSoup, parses phone numbers with phonenumbers, and writes results with pandas + openpyxl. A JSON cache reduces repeated searches. Agent pattern: Goal -> Search/Act -> Validate/Observe -> Decide -> Retry or Save. I added a controlled loop so unresolved fields can use alternate strategies, stop immediately when verified, and stop after a limited number of attempts.

3. Data Quality, Hallucination Control, and Human QA

Search results were not accepted blindly. Rules prefer official domains, reject retailers/directories/look-alike or regional sites, rank customer-service numbers above warranty/fax/media numbers, distinguish headquarters from distribution offices, preserve source information, and recheck suspicious values. Questionable records were corrected only when stronger official evidence existed; otherwise they stayed UNKNOWN. This human-in-the-loop approach prioritized defensible data over filling every field.

4. Challenge: Streamlit User Interface

I added app.py with Streamlit so an evaluator can use the project without running Python commands. The UI provides Final Dataset, Try the Agent, and How It Works views. The app was deployed to Streamlit Community Cloud and is publicly accessible.

5. Extra Challenge: AWS Serverless Architecture

Terraform Infrastructure as Code was used to provision AWS resources instead of creating them manually. The deployment included DynamoDB for NoSQL storage, AWS Lambda for serverless compute, IAM permissions, AWS CLI authentication, and boto3 for Python-to-AWS integration. The validated dataset was uploaded to table ai-data-augmentor-companies; 50 records were loaded. Lambda function ai-data-augmentor-api supports health/list/get operations. The real deployment returned a 200 health response and successfully retrieved the Patagonia record from DynamoDB. Local Lambda tests also returned PASS before deployment.

6. Super Extra: GitHub Actions, S3, and CloudFront

GitHub Actions workflow .github/workflows/ci.yml runs on push/pull request to main. It installs Python dependencies, checks syntax, runs the local Lambda test, initializes Terraform, and runs terraform validate. Both Python and Terraform jobs completed successfully with green status. AWS deployment remains intentionally manual. Terraform also provisions a private S3 bucket, public-access protection, CloudFront Origin Access Control, bucket policy, and CloudFront distribution. The final Excel file was uploaded to S3 and successfully downloaded through CloudFront while the S3 bucket remained private.

7. End-to-End Architecture

Starter CSV -> Python augmentor -> Search -> Validate -> Loop/Retry -> Human QA -> Excel. Excel -> Streamlit UI. Excel -> DynamoDB -> Lambda. Excel -> Private S3 -> CloudFront CDN. GitHub Push -> GitHub Actions -> Python/Lambda/Terraform validation.

8. Build, Debugging, and Operational Lessons

I moved from an earlier Tavily prototype to the free DDGS approach, fixed Git/GitHub issues caused by accidentally staging the large .terraform provider directory, kept .terraform.lock.hcl while excluding generated Terraform files/state, resolved an AWS Python dependency issue, and worked through PowerShell payload formatting for Lambda tests. I also added an AWS zero-spend budget alert and kept deployment manual to reduce cost and change risk.

9. Reflection and Lesson Learned

This assignment taught me that a useful AI/data agent is not simply an API call that returns a value. The value comes from the control logic around the tools: source selection, validation, confidence handling, retries, stopping conditions, exception handling, provenance, and human review. I also learned how a local Python workflow can be expanded into a complete cloud solution with a UI, serverless API, NoSQL storage, Infrastructure as Code, automated CI checks, and CDN delivery. My main takeaway: good automation should know when to trust a result, when to retry, when to ask for human review, and when to return UNKNOWN.

Public Access / Submission Evidence

GitHub Repository: <https://github.com/aobuhovskiy/Aleks-Obuhovskiy-AI-Data-Augmentor>

Live Streamlit App: <https://aleks-obuhovskiy-ai-data-augmentor-n6ex28xpz3tdlfgghhwbst.streamlit.app/>

CloudFront Excel: <https://d2uhh4vke00btl.cloudfront.net/augmented-companies.xlsx>